

Clase 021 — Criptografía: conceptos fundamentales e intuición

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Ferguson, Schneier & Kohno, Cryptography Engineering* 🕒 Duración estimada: **120 min** · Nivel: **Fundamentos**

🎯 Objetivo

Construir la intuición criptográfica que sostiene TLS, firmas, autenticación y almacenamiento seguro de contraseñas. Al terminar entenderás cifrado simétrico y asimétrico, funciones hash, HMAC, firmas digitales y por qué "nunca inventes tu propia cripto". No se trata de matemáticas avanzadas, sino de saber usar las primitivas correctas.

📖 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Distinguir** cifrado simétrico de asimétrico y sus usos.
2. **Explicar** funciones hash, colisiones y propiedades de seguridad.
3. **Diferenciar** hashing de contraseñas (bcrypt/argon2) de hash genérico.
4. **Describir** HMAC, firmas digitales e intercambio de claves.
5. **Aplicar** primitivas correctas con una librería confiable.

📚 Temas

#	Tema	Por qué importa
1	Terminología	Confidencialidad, integridad, autenticidad
2	Cifrado simétrico	AES, modos, IV/nonce
3	Cifrado asimétrico	RSA, ECC, par de claves
4	Funciones hash	SHA-2/3, colisiones
5	Hash de contraseñas	bcrypt, scrypt, argon2, sal
6	HMAC y MAC	Integridad autenticada
7	Firmas digitales	No repudio
8	Intercambio de claves	Diffie-Hellman, PKI

📖 Definiciones y características

- **Cifrado simétrico:** misma clave cifra y descifra (AES). Clave: rápido; problema = distribuir la clave de forma segura.

- **Cifrado asimétrico:** par pública/privada (RSA, ECC). Clave: resuelve la distribución; más lento, se usa para intercambiar claves simétricas.
- **Función hash criptográfica:** mapeo unidireccional resistente a colisiones (SHA-256). Clave: verifica integridad; MD5 y SHA-1 están rotos.
- **Sal (salt):** valor aleatorio único por contraseña. Clave: impide tablas rainbow y que hashes iguales delaten contraseñas iguales.
- **HMAC:** MAC basado en hash + clave secreta. Clave: garantiza integridad **y** autenticidad, a diferencia de un hash simple.
- **Firma digital:** cifrar un hash con la clave privada. Clave: prueba autoría e integridad (no repudio).

Herramientas y preparación

Usa Python con la librería **cryptography** (de PyCA) y `hashlib`. Para contraseñas, `bcrypt` / `argon2-cffi`. Instala en tu venv:

```
pip install cryptography bcrypt argon2-cffi
```

También `openssl` en línea de comandos para experimentar. **Regla de oro:** usa librerías establecidas, nunca implementaciones caseras de primitivas.

Laboratorio guiado

1. Hash e integridad:

```
python import hashlib print(hashlib.sha256(b"mensaje").hexdigest())
```

Cambia un byte del mensaje y observa el efecto avalancha. 2. **Por qué MD5/SHA-1 no valen.** Investiga colisiones conocidas y razona por qué no deben usarse para integridad de seguridad. 3. **Cifrado simétrico autenticado** con AES-GCM (vía `cryptography`):

```
python from cryptography.hazmat.primitives.ciphers.aead import AESGCM key =
AESGCM.generate_key(bit_length=256) aes = AESGCM(key); nonce = b"\x00"*12 ct =
aes.encrypt(nonce, b"secreto", None) print(aes.decrypt(nonce, ct, None))
```

Nota: en producción el nonce debe ser único por mensaje. 4. **Hash de contraseñas con sal:**

```
python import bcrypt h = bcrypt.hashpw(b"Contrasena1", bcrypt.gensalt())
print(bcrypt.checkpw(b"Contrasena1", h))
```

Compara: hashear una contraseña con SHA-256 "pelado" es inseguro. 5. **HMAC** para integridad autenticada de un mensaje con clave compartida. 6. **Claves asimétricas.** Genera un par RSA con `openssl` y firma/verifica un archivo:

```
bash openssl genrsa -out priv.pem 2048 openssl rsa -in priv.pem -pubout -out pub.pem openssl
dgst -sha256 -sign priv.pem -out firma archivo openssl dgst -sha256 -verify pub.pem -signature
firma archivo
```

Ejercicios

1. Explica con un ejemplo cuándo usarías cifrado simétrico y cuándo asimétrico (y por qué se combinan).
2. Da tres propiedades que debe cumplir una función hash criptográfica.
3. Justifica por qué SHA-256 no es adecuado para almacenar contraseñas y qué usar en su lugar.
4. Describe cómo una firma digital aporta integridad, autenticidad y no repudio.
5. Explica el papel del IV/nonce y qué ocurre si se reutiliza en algunos modos.
6. Investiga qué es Diffie-Hellman y cómo permite acordar una clave sin transmitirla.

Reto verificable

Implementa `securebox.py`, una herramienta que cifre y descifre un archivo con AES-GCM (clave derivada de una contraseña mediante un KDF adecuado como `scrypt/argon2` con sal), verificando integridad al descifrar. Debe fallar de forma clara si el archivo fue manipulado o la contraseña es incorrecta.

Criterio de aceptación: un archivo cifrado y luego descifrado con la contraseña correcta se recupera idéntico (mismo SHA-256); alterar un byte del cifrado o usar contraseña errónea produce un error de autenticación explícito, no datos corruptos silenciosos. Usa una librería establecida (PyCA cryptography), sin cripto casera.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Almacenar contraseñas con SHA-256 sin sal	Vulnerable a rainbow tables y GPU cracking. Usa <code>bcrypt/argon2</code> con sal.
Reutilizar el nonce en AES-GCM	Rompe la seguridad del cifrado. Genera un nonce único por mensaje.
Implementar AES "a mano"	Errores sutiles = inseguro. Usa librerías auditadas.
Usar MD5/SHA-1 para firmas o integridad	Están rotos por colisiones. Migra a SHA-256/SHA-3.
Cifrar sin autenticar (solo AES-CBC)	Permite manipulación. Usa modos autenticados (GCM) o "encrypt-then-MAC".

Preguntas frecuentes

? ¿Por qué "no inventes tu propia cripto"? Porque las primitivas son fáciles de implementar mal de formas invisibles pero catastróficas (timing, padding, nonces). Las librerías establecidas han sido auditadas por expertos durante años.

? ¿Cifrado o hashing para contraseñas? Ninguno de los dos "a secas": se usa un **hash de contraseñas** lento y con sal (`bcrypt/scrypt/argon2`), diseñado para resistir fuerza bruta.

? ¿RSA está obsoleto? No, pero la criptografía de curva elíptica (ECC) ofrece la misma seguridad con claves más pequeñas y es preferida hoy. Ambos conviven.

? **¿Qué es la criptografía post-cuántica?** Algoritmos resistentes a computadores cuánticos, que amenazan RSA/ECC. El NIST ya estandariza esquemas PQC; es un tema emergente que solo introducimos aquí.

Referencias

- Ferguson, Schneier & Kohno, *Cryptography Engineering*.
- PyCA cryptography — <https://cryptography.io/>
- NIST Cryptographic Standards — <https://csrc.nist.gov/projects/cryptographic-standards-and-guidelines>
- OWASP Password Storage Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Siguiente clase

Clase 022 - Docker y contenedores para laboratorios de seguridad